

Predict, Measure, Update: How Neural Networks Learn

Lecture 4 essay — Outline

The Core Idea

- Every network ever trained — a two-parameter line fit or a frontier language model — learns by repeating three steps: **make a prediction, measure how wrong it was, adjust the weights** so that next time it is a little less wrong. *Predict, measure, update*. Everything else is detail on those verbs, and most of it is detail on the third
- Order of the essay: what goes in and out → the **loss function** (“how wrong” as a single number) → **gradient descent** (which way to move) → batching and backpropagation (feasibility at scale) → the loop in PyTorch → how the same loop trains a language model, and what changes when you fine-tune instead of starting from scratch

I. Inputs and Outputs

- A deep learning model is general in a way classical models are not: image, sound, or text in; a category, a number, or a different modality out (speech → text, caption → image). The model never sees pixels or words — only vectors of numbers
- Two translation steps bracket every model. **Encoding**: real-world input → mathematical object (image → grid of brightness values; sentence → token IDs → embeddings). **Decoding**: output vector → something usable (ten scores → “ankle boot”; a distribution → the next word). The scales need not match: 0–255 in, a 0/1 verdict out
- Call the guess $\hat{y}$ and the truth y . Training is about the gap between them

II. Measuring Performance: The Loss Function

- A **loss function** collapses a whole set of predictions into one non-negative number: near zero is good, large is bad, negative never happens. In supervised learning y comes from the answer key and the loss is summed over every example
- **Mean squared error**, the workhorse for regression:

$$\mathcal{L}_{\text{MSE}} = \sum_i (\hat{y}_i - y_i)^2.$$

Subtract, square, add up. Why square: of necessity, errors must be positive — overshooting by 7 and undershooting by 7 are equally wrong and must not cancel; of taste, squaring punishes large errors disproportionately — 70^2 is a hundred times 7^2 , not ten. A cubic keeps the sign and is not a candidate. The cost: outliers get amplified, so filter extreme points (beyond ≈ 3.5 standard deviations) first. The gentler alternative is **mean absolute error**, $\sum_i |\hat{y}_i - y_i|$. MSE vs. MAE is a low-priority choice: when training fails, the loss function is rarely the reason — check the learning rate first

- **Cross-entropy**, the loss for classification:

$$\mathcal{L}_{\text{CE}} = - \sum_i p_i \log \hat{p}_i,$$

with p the true distribution and $\hat{p}$ the predicted one. Same behavior as MSE: close costs little, far costs a lot (10% sun on a rainy day is cheap; 95% sun costs about 3.5). Preferred over $1 - \hat{p}$ because it falls out of information theory rather than being invented. It matters here because *it is the loss language models are trained on*: next-token prediction is classification over fifty thousand categories instead of ten

III. The Update Step: Gradient Descent

- Training, precisely: the loss depends on the weights, and training is the search for the weights that minimize it. “Line of best fit” is this phrase — *best* means minimum loss — for two parameters instead of billions. The search algorithms are **optimizers**, and every modern one (Momentum, RMSProp, Adam, AdamW) is gradient descent with sprinkles
- **The gradient**: the derivative in every dimension at once — a vector whose j th entry is the slope with respect to the j th weight:

$$\nabla_{\theta}\mathcal{L} = \left[\frac{\partial\mathcal{L}}{\partial\theta_1}, \frac{\partial\mathcal{L}}{\partial\theta_2}, \dots, \frac{\partial\mathcal{L}}{\partial\theta_n} \right].$$

Core principle: *the sign of the slope tells you the direction of increase*, so to lower the loss, step against it. Left of the minimum the slope is negative and you step right; right of it, positive and you step left. It holds in every dimension independently, so the same rule works for one weight or a hundred billion. One step:

$$\theta \leftarrow \theta - \eta \nabla_{\theta}\mathcal{L}(\theta).$$

- **The loss landscape**: be clear which graph is drawn. Lecture one plotted data against predictions; this lecture’s axes are the *weights* and the height is the *loss*. For $y = ax + b$ the loss is a surface over a plane and training is a ball rolling toward the lowest valley; $y = ax$ gives a curve. The classic worry: zero slope marks any minimum, local or global, so the ball can settle in a shallow dip. It matters less than intuition suggests — momentum and the noise of small batches jostle the ball out, and in very high dimension true traps are rarer than they look on a 3-D plot
- **The learning rate η** : a scalar that scales how far each step moves. The most consequential knob in training and the hardest to set by reasoning. Too small: microscopic steps, loss flat for hundreds of iterations (“a million years”). Too large: each step overshoots to the far slope, and if the overshoot grows the loss climbs. In between: overshoots that shrink, and convergence anyway. Read the regime off the loss curve — what you want is a loss that is, on the whole, going down. Refinements: **learning-rate decay** shrinks η late in training; **momentum** carries velocity from earlier steps; AdamW manages per-parameter scaling on its own
- **The foggy mountain**: you want the valley but can only feel the ground under your feet, and measuring steepness needs a slow, expensive instrument, so you take a reading, walk downhill a while, and read again. The mapping is exact — you are the algorithm; the path is the sequence of weights; the steepness is the gradient; the instrument is differentiation; the distance between readings is learning rate $\times$ gradient. The fog is why nobody sees the whole landscape: in millions of dimensions there is no map, only the slope where you stand. Knowing the minimum in advance is like finding the largest element of a 50,000-entry array without looking through it

IV. Making It Feasible: Batches and Backpropagation

- Two practical questions: how much data per gradient reading, and how the gradient is actually computed
- **Batch size**. **Batch gradient descent** averages over the whole dataset — the true gradient, a clean direct path. **Stochastic gradient descent** (SGD) uses one example — noisy, zigzagging, each gradient a rough estimate. **Mini-batch** sits in between (32, say) and is what almost everyone runs; powers of two for GPU memory layout. The trade-off is efficiency against accuracy: computing the gradient is the bottleneck, and fifty one-second steps that point roughly right beat three ten-minute steps that point exactly right. The noise also jostles the ball — one reason local minima are less of a problem in practice. Lecturer’s preference: 32, 64, or 128; in most research and production settings hundreds to thousands are standard, and batch size 1 is usually a memory workaround. The binding constraint: the batch must fit on the GPU

- Vocabulary: in lecture “epoch” meant one update; standard usage reserves **epoch** for a full pass through the dataset and calls each update a **step** or **iteration**
- **Backpropagation**: in practice one line, `loss.backward()`. What is wanted is $\partial\mathcal{L}/\partial\theta$ for every weight. A network is a nested composition — weighted sum, activation, weighted sum, activation — so the derivative is the chain rule applied repeatedly:

$$\frac{\partial\mathcal{L}}{\partial\theta^{(1)}} = \frac{\partial\mathcal{L}}{\partial h^{(L)}} \cdot \frac{\partial h^{(L)}}{\partial h^{(L-1)}} \cdots \frac{\partial h^{(2)}}{\partial h^{(1)}} \cdot \frac{\partial h^{(1)}}{\partial\theta^{(1)}}.$$

Each factor is one layer’s derivative with respect to the layer before it. Each layer’s contribution depends on the layers *after* it, and only the last layer’s derivative — the loss — is known at the outset, so you work backwards from output to input. Hence the name. Efficient because intermediates are shared: the derivative through layer 5 is needed for layers 4, 3, 2, 1. Backpropagation is **dynamic programming** over a directed acyclic graph — compute each intermediate once, store it, look it up. The calculus needed is small (any intro series plus 3Blue1Brown’s third and fourth videos)

- Honest caveat: none of this comes with guarantees. That noise helps, that the ball escapes dips, that the loss falls — empirical regularities, not theorems. The discipline is to watch the loss and the validation accuracy; accuracy at the chance baseline (25% on four classes) means something is broken, and the learning rate is the first suspect

V. The Loop in Code

- Setup: a hosted GPU on Colab (a free T4 suffices), `torch` and `torch.nn`, FashionMNIST (ten clothing classes, sibling of MNIST). A `DataLoader` handles batching, shuffling, and parallel loading — a convenience, not a concept. First sanity check is shape: a batch is `[64, 1, 28, 28]` — batch, channels, height, width — with 64 labels; dimension mismatches are the first thing to look for
- Model: `Linear(784→512)`, `ReLU`, `Linear(512→512)`, `ReLU`, `Linear(512→10)`, with $784 = 28 \times 28$ the flattened image; `forward` flattens, applies the stack, returns **logits**. In real projects you import a pretrained architecture
- The training loop reads best as instructions to objects:

```
loss_fn = nn.CrossEntropyLoss()           # classification -> cross-entropy
optimizer = SGD(model.parameters(), lr=1e-3) # optimizer: weights + learning rate

model.train()                             # model, switch to learning mode
for batch, (X, y) in enumerate(dataloader):
    X, y = X.to(device), y.to(device)      # data, move onto the GPU
    pred = model(X)                        # model, make your prediction
    loss = loss_fn(pred, y)                # loss function, score vs. truth
    loss.backward()                         # loss, compute the gradient (backprop)
    optimizer.step()                       # optimizer, move the weights one step
    optimizer.zero_grad()                  # optimizer, clear the gradient
```

The four inputs — data, model, loss function, optimizer — are the four boxes in the training diagram; the body is the three steps: predict, compute the loss, compute the gradient and step. The one non-obvious line is `zero_grad`: PyTorch accumulates gradients by default

- Validation is the same loop with learning turned off: `model.eval()`, wrap in `torch.no_grad()`, accuracy = fraction with `pred.argmax(1) == y`. (The notebook calls it the test loop; it is really validation.) Five epochs took validation accuracy from 48% to 53% with validation loss falling — unimpressive, but the curve heads the right way; print both losses
- Housekeeping: `torch.save(model.state_dict(), path)` and `model.load_state_dict(torch.load(path))`; save to a mounted Drive, because Colab wipes local

storage when the runtime dies

VI. Training a Language Model

- Everything so far assumed an answer key; the internet has none. The answer is **self-supervised learning**, between supervised and unsupervised: the model labels the data itself by predicting one part of the input from another. Hide a word, predict it from what came before; an n -token sentence yields $n - 1$ examples with built-in answers. Pipeline: the internet $\rightarrow$ basic filtering $\rightarrow$ next-token problems by the trillion. No labelers, though frontier labs invest heavily in data triage and upsampling
- But what is the ground truth? “The plate was a beautiful shade of ____” — the true distribution (30% gray, 50% blue) is unknowable. The surrogate: treat the token that appeared as 100% correct, a **one-hot** target. Cross-entropy against a one-hot collapses to a single term,

$$\mathcal{L}_t = -\log \hat{p}(x_t^*),$$

the negative log-probability assigned to the token that came next. Each individual target is wrong, but at internet scale the errors average out — exactly as Monte Carlo sampling estimates a distribution from draws. The cost: it rewards confidence, pushing the model toward certainty on every example — one contributor to the **miscalibration** of trained language models

- The lecture-2 picture is now complete: that lecture covered post-training (SFT, RLHF, RLVR), this one pre-training and the mechanics of training. The only component still a black box is attention

VII. Fine-Tuning, and the Art of Practice

- You will almost never train from scratch: frontier pre-training costs billions. Exception: small **model organisms** for interpretability research, with the caveat that what holds for a tiny model need not hold at scale
- **Fine-tuning** = take an existing model’s final weights and train a few more epochs on your own data. Not a different kind of training — the same loop with a different starting point: weights that already understand language or images instead of random ones. Also called **transfer learning**; why 5,000 math problems can teach a model math — but only a model that already reads English (the brilliant exchange student who cannot read the word problems)
- Clearest case: the messy middle — a hospital’s 10,000 clinical notes dense with local acronyms, where a general model underperforms and a from-scratch model memorizes or is too weak. The alternative worth knowing is **RAG**: leave the model alone and hand it the relevant notes at query time
- Two risks: **overfitting**, the familiar one; and **catastrophic forgetting**, subtler and unsolved — learning the new thing degrades the old things. The field attacking it is **continual learning**; its lack of a clean solution is a standard argument in the skeptic’s case
- **Practical guidelines**, hard-won rather than derived: *the learning rate is the biggest lever* and nearly impossible to reason out (10^{-3} to 10^{-15}) — ask mentors and PIs; when fine-tuning, start from the *final*, decayed rate the pretrained model ended on. Use AdamW. Start the batch size at 1 and grow it. Deliberately overfit first: underfitting tells you nothing, overfitting proves the pipeline works and you can pull back. Monitor validation loss with Weights & Biases, not print statements. Save weights every epoch to permanent storage. Inference memory $\approx$ twice the parameter count in GB (a 7B model wants 14–15 GB in half precision, far less at 4-bit)
- **Hyperparameters**: settings fixed before training that training cannot learn — the learning rate (how training proceeds), the number of layers and neurons (what is being trained). One meta-level above the loop. **Grid search** tries every combination and the arithmetic is unkind: 10 learning rates $\times$ 3 depths $\times$ 3 widths = 90 runs, at five hours each 450 hours, exponential in each added dimension. **Random search** (ten of the 90) is, naive as it sounds, the practical default. **Bayesian optimization** (Meta’s labs) and Optuna are usually overkill

One Loop, Billions of Weights

- **Predict:** encode the input, run it through the network, decode a guess. **Measure:** compare to the truth with a loss — squared error for numbers, cross-entropy for categories, cross-entropy against a one-hot next token for language. **Update:** compute the gradient by backpropagation, scale it by the learning rate, step against it, repeat — one example at a time or a thousand, from random weights or from a model someone else trained
- The four foundation lectures — ML basics, post-training, neural networks, training — are the complete picture, with attention the single remaining black box. Do not keep learning horizontally. The foundation is laid; go straight into research